arXiv:2610.22984v1 [cs.CL] 16 Oct 2026

Self-Evolving Agents Can Repair Failures but Struggle to Generalize from Them

Weixiang Zhao[1*], **Yichen Zhang**[1,2*], **Yanyan Zhao**[1], **Jinming Bai**[1], **Bin Liang**[3], **Ruifeng Xu**[2], **Bing Qin**[1], **Ting Liu**[1]
[1]Harbin Institute of Technology, Harbin, China
[2]Harbin Institute of Technology, Shenzhen, China
[3]The Chinese University of Hong Kong
{wxzhao, yiczhang}@ir.hit.edu.cn

Abstract

Experience-driven self-evolution offers a promising path for large language model (LLM) agents to continually improve through interaction. Yet existing approaches often rely on successful trajectories or external evaluation signals, leaving unclear whether agents can evolve when such positive anchors are unavailable. We study this question through *Cold-Start Failure-Only Evolution* (CFOE), where agents start without accumulated experience and learn only from failed interactions without privileged guidance. Across three self-evolving paradigms, multiple LLM backbones, and four interactive environments, we uncover a consistent *repair–generalization gap*: failure-derived experience can help repair previously failed tasks, but rarely yields sustained improvements on unseen tasks. Further analysis shows that agents often misinterpret failures, while even verified-correct experience remains difficult to transfer. Relaxing CFOE reveals that verifier feedback and additional interaction provide limited gains, whereas successful trajectories substantially improve transfer. These findings suggest that current self-evolving agents are better at repairing observed failures than building generalizable capabilities from failure.

1 Introduction

With the arrival of the era of experience, large language model (LLM) agents are increasingly expected to improve through their own interactions (Silver and Sutton, 2025). Self-evolving agents have therefore emerged as a promising paradigm that continually accumulates and reuses interaction experience (Dou et al., 2025; Cai et al., 2026; Bell et al., 2025; Gao et al., 2026).

Yet existing self-evolution studies typically assume access to positive learning signals, such as successful trajectories, verifier feedback, or already useful experience (Ouyang et al., 2026; Yang et al., 2026a,b). In challenging open environments, however, these signals may be scarce precisely when adaptation is most needed: agents can start without prior experience, repeatedly fail on unfamiliar tasks, and receive little guidance on how to improve. This regime provides a critical test of autonomous self-evolution—whether agents can bootstrap new capabilities from failure itself, rather than relying on success as a prerequisite for learning.



Figure 1: *Cold-Start Failure-Only Evolution* on a backbone-specific, failure-dense online task stream. Starting from an empty external state, agents learn only from failed interactions, without access to ground-truth answers, expert solutions, or verifier feedback. We investigate whether existing self-evolving agents can turn failure-derived experience into transferable capability under these conditions.

To study this problem, we introduce *Cold-Start Failure-Only Evolution* (CFOE), a setting that isolates self-evolution from failures themselves (Figure 1). Agents start from an empty external state and evolve over a failure-dense online task stream. Successful executions may occur, but they are used only for evaluation and never for reflection or up-

*Equal contribution

dating. Verifier feedback, reward scores, ground-truth answers, and expert solutions are also hidden from the agent and updater. Thus, self-evolution is driven exclusively by failed interactions, requiring agents to diagnose failures and extract reusable experience without privileged guidance.

We first investigate **(RQ1)** *to what extent agents can learn from failures under Cold-Start Failure-Only Evolution, and whether such learning generalizes to unseen tasks* (§3). We evaluate three representative paradigms—ReasoningBank for reasoning-memory evolution (Ouyang et al., 2026), SkillOpt for skill evolution (Yang et al., 2026b), and Evo-Harness for runtime harness evolution (Wei et al., 2026)—across multiple LLM families and four interactive environments. Across frameworks, models, and environments, we observe a consistent *repair–generalization gap*: failure-derived experience can partially recover previously failed tasks, yet these gains rarely translate into sustained improvements on unseen tasks and may even degrade held-out performance. Extending evolution to much longer horizons does not close this gap.

We then investigate **(RQ2)** *what prevents failure-derived experience from becoming transferable capability* (§4). By tracing trajectory interpretation, failure attribution, and experience consolidation, we identify recurring errors such as false-success attribution, misattribution, distorted consolidation, and harmful guidance. These results reveal a diagnosis bottleneck: agents may turn plausible but unsupported interpretations of failures into persistent experience. Yet diagnosis alone is not the whole problem. Even after retaining only verified-correct experience, source-task recovery improves while held-out performance remains unchanged, showing that correct experience can still remain overly tied to the source trajectory and fail to transfer.

Finally, we turn to **(RQ3)** *what learning conditions are missing for effective experience transfer* (§5). We relax CFOE by allowing successful trajectories, exposing verifier feedback, or permitting additional interaction during reflection. Verifier feedback and extra interaction both improve failure understanding, but yield limited transfer gains. In contrast, successful interactions produce substantially stronger held-out improvements, suggesting that positive behavioral evidence provides information that failure analysis alone cannot easily recover. Failures reveal what went wrong, while successful trajectories show what reliably works and how useful actions compose into complete procedures.

Overall, our study provides a controlled evaluation of self-evolution before success. We show that current agents can extract useful local corrections from failed interactions, but struggle to turn them into transferable capability (§3). We further trace this limitation to unreliable failure interpretation and the limited transferability of even correct experience (§4), and show that successful behavioral evidence provides substantially stronger support for transferable evolution than richer failure diagnosis alone (§5). These findings suggest that current self-evolving agents are better at *repairing observed failures* than at *building generalizable capabilities from failure*, highlighting a fundamental challenge for autonomous learning in open environments.

2 Cold-Start Failure-Only Evolution

We formulate an online self-evolving agent by a fixed backbone policy π_θ, an evolving external state S_t, an updater U, and a verifier V. Given a sequential task stream $\{x_t\}_{t=1}^{T}$, the agent executes each task with its current external state,

$$\tau_t \sim \pi_\theta(\cdot \mid x_t, S_{t-1}),$$

where θ remains fixed throughout evolution. The updater then modifies S_t based on the interaction trajectory. Existing self-evolving agents mainly differ in the representation of S_t—e.g., reasoning memory, agent skills, or runtime harness state—and in how U updates it.

Our *Cold-Start Failure-Only Evolution* (CFOE) setting imposes three constraints. Evolution starts from an empty external state, only failed trajectories are used for updating, and all verifier feedback is hidden from both the agent and the updater. Let $y_t = V(x_t, \tau_t) \in \{0, 1\}$ denote the verifier judgment available only to the experimental controller. Then

$$S_t = \begin{cases} S_{t-1}, & y_t = 1, \\ U(S_{t-1}, \tau_t), & y_t = 0. \end{cases}$$

Thus, successful executions may occur occasionally but never contribute to evolution, while success/failure labels, reward scores, verifier explanations, ground-truth answers, and expert solutions remain inaccessible throughout the process.

3 Failure-Only Evolution Repairs but Does Not Generalize

We begin by empirically answering **RQ1**: to what extent can agents learn from failures, and whether

such learning generalizes to unseen tasks.

3.1 Experimental Setup

Self-Evolving Agents We study three paradigms with frozen backbones: reasoning-memory evolution with **ReasoningBank** (Ouyang et al., 2026), skill evolution with **SkillOpt** (Yang et al., 2026b), and runtime harness evolution with **Evo-Harness** (Wei et al., 2026). In all three frameworks, self-evolution arises from the accumulation and refinement of external artifacts. Framework details are provided in Appendix A.

Backbone Model. We conducted experiments with 10 LLM backbones that span multiple model families, parameter scales, and architectures. Across all three self-evolution frameworks, we include a wide spectrum of dense variants of Qwen3.5 family (4B, 9B, and 27B) (Qwen Team, 2026) and Gemma 4 family (Gemma-4-12B-it and Gemma-4-31B-it) (Gemma Team, 2026), as well as the mixture-of-experts models Qwen3.5-35B-A3B and Gemma-4-26B-A4B-it. We further extend the evaluation to DeepSeek-V4-Pro (DeepSeek-AI, 2026), GLM-5.1 (GLM-5-Team et al., 2026) and GPT-5.6 Sol (OpenAI, 2026a) in selected frameworks, given the cost of the held-out evaluation at multiple evolution checkpoints.

Environment & Benchmark. Our experiments cover household embodied task execution, long-horizon interactive tool use, online shopping, and web navigation across four *quality-verified* environments: **ALFWorld** (Shridhar et al., 2021), **τ^2-Bench-Verified** (Barres et al., 2026; Cuadron et al., 2026), **WebArena-Verified** (El Hattami et al., 2025) (Shopping, Shopping Admin, Reddit, and GitLab), and **WebShop** (Yao et al., 2022). These environments require multi-step interaction and provide a diverse testbed for long-horizon agentic adaptation. Benchmark details can be found in the Appendix B.

Implementation Details. Closed-source and large-scale open-weight backbones are accessed via official APIs, while other open-weight models are deployed locally with vLLM (Kwon et al., 2023) on NVIDIA H100 GPUs. We adopt the recommended configurations and evaluation protocols for each agent framework and benchmark. For SkillOpt, we adapt the offline setup to an online setting to align with our self-evolution protocol. Further configurations are provided in the Appendix C.1.

Failure-Dense Online Evolution Streams Construction. Given the inherent differences in initial task-solving capabilities across model backbones, we first construct a challenging task stream for each backbone on each benchmark independently for online self-evolution. Each task involved in the constructed stream has resulted in a consistent failure in 3 independent preliminary rollouts with the corresponding backbone, inducing a high density of failures during subsequent online self-evolution. Details of the stream construction procedure are provided in the Appendix C.2.

Online Evolution from Failures Alone Each run starts with no accumulated memory or skills and a fresh harness state. The agent processes the challenging task stream sequentially, using failed trajectories to reflect and evolve its memory, agent skills, or runtime harness state. Occasional successes are recorded only for final reporting; the agent moves directly to the next task without using them for reflection or self-evolution updates.

Ground Truth & Reward Blind The controller uses verifier judgments to collect failed trajectories, but neither the agent nor updater receives outcome labels, rewards, verifier explanations, ground-truth answers, or expert solutions. Agents are also unaware of this failure-only selection. Details of feedback setting appear in Appendix C.3.

3.2 Results and Analysis

Figure 2 presents CFOE on ALFWorld and τ^2-Bench-Verified across the three frameworks. Each benchmark pairs held-out curves with source-task recovery bars for the same backbones. Results on WebArena-Verified and WebShop are provided in Appendix D.1.

Cold-Start Failure-Only Evolution yields limited held-out gains and can even degrade performance. We evaluate successive checkpoints on a fixed unseen held-out set disjoint from the online stream. Agent artifacts remain unchanged throughout evaluation, with no further self-evolution on held-out tasks (Appendix C.4). The held-out curves in Figure 2 show limited held-out gains, with some configurations even declining as experience accumulates. SkillOpt exhibits smaller held-out fluctuations than ReasoningBank and Evo-Harness. This relative stability may stem from its validation-based acceptance mechanism, which retains candidate skills only when they improve validation

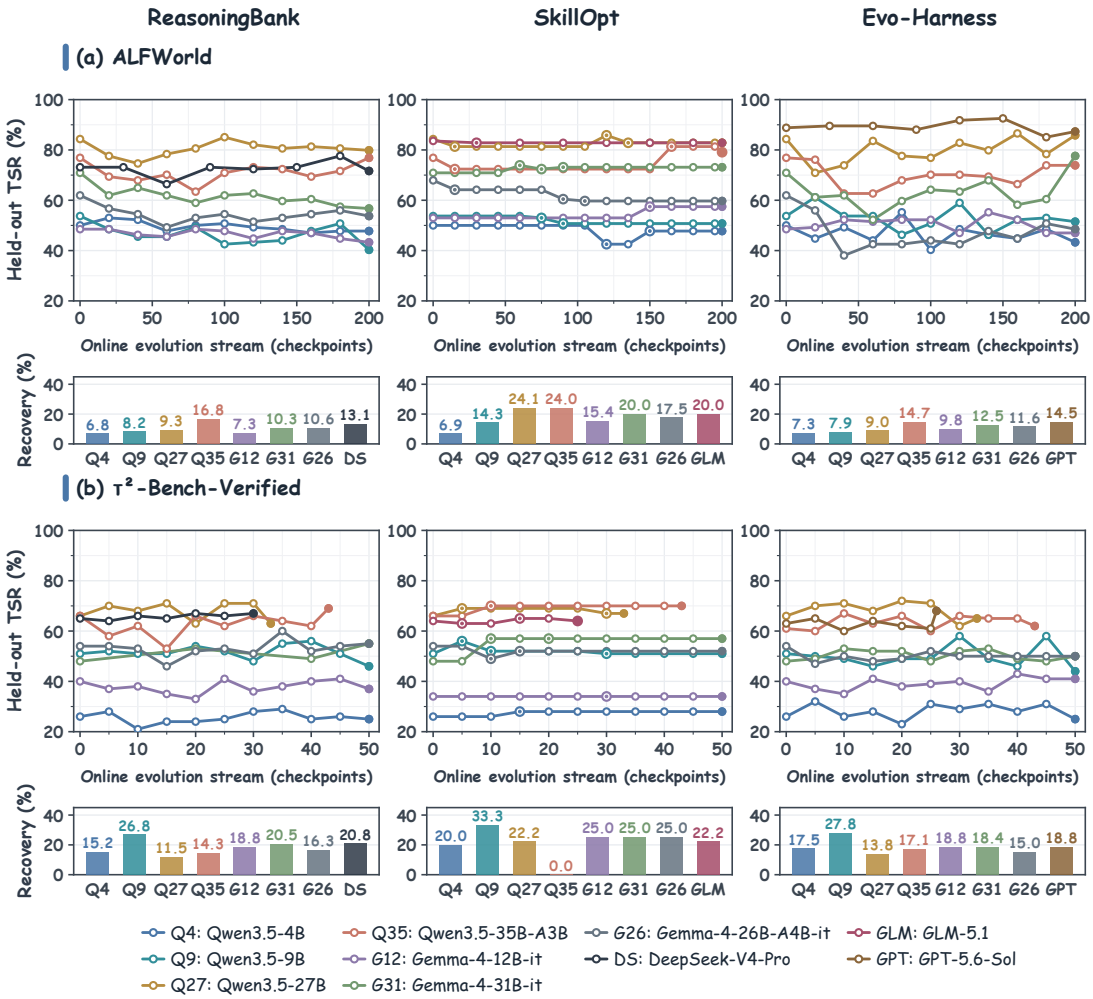


Figure 2: Held-out task success rate (TSR) and source-task recovery during cold-start failure-only evolution on (a) ALFWorld and (b) τ^2-Bench-Verified. Results are shown for ReasoningBank, SkillOpt, and Evo-Harness, with eight backbone models per framework. Some τ^2-Bench-Verified failure-dense streams contain fewer than 50 tasks.

performance. Even so, gains on held-out tasks remain limited.

Failure-derived experience supports partial recovery of original failed tasks. After each effective update, we retry its failed source tasks once from their initial states, using the experience state immediately after that update without further reflection or updates. For these two benchmarks, recovery is the percentage of successful retries over the entire online run (Appendix C.5). The recovery bars in Figure 2 show partial source-task recovery, with rates varying across backbones, frameworks, and environments. Qwen3.5-27B with SkillOpt recovers more ALFWorld source tasks than Qwen3.5-4B, yet its held-out performance still declines. These results show that improvements in source-task recovery do not reliably carry over to held-out tasks.

3.3 Long-Horizon Online Self-Evolution

To examine whether these limitations persist as online self-evolution scales to a longer horizon, we extend the ALFWorld task stream to 800 episodes. All 800 tasks have failed in three independent preliminary rollouts with the corresponding backbone and no accumulated experience. Figure 3 pairs held-out curves and recovery performance for Qwen3.5-4B, Qwen3.5-27B, and GLM-5.1 across all three frameworks.

The limited held-out gains persist over the scaled evolution horizon. Held-out performance declines further for Qwen3.5-27B with Evo-Harness and plateaus for Qwen3.5-4B with SkillOpt, despite partial source-task recovery in both configurations. These long-horizon results are consistent with our earlier observations: failure-derived experience supports partial source-task recovery, but continued accumulation does not yield sustained held-out

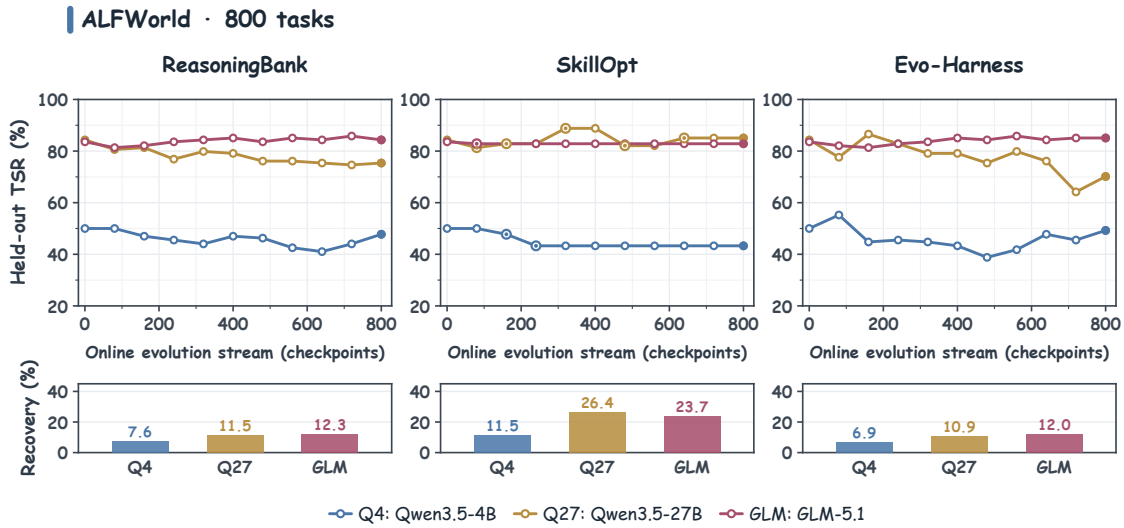


Figure 3: Held-out TSR and source-task recovery during cold-start failure-only evolution over 800 ALFWorld tasks. Results are shown for Qwen3.5-4B, Qwen3.5-27B, and GLM-5.1 across ReasoningBank, SkillOpt, and Evo-Harness.

improvements.

4 Causes of Limited Experience Transfer

To answer **RQ2**—what prevents agents from turning failure-derived experience into transferable capability—we conduct two complementary analyses. We first examine the *faithfulness of experience formation*, tracing how failed trajectories are interpreted, attributed, and consolidated into reusable experience (§4.1). We then ask whether, even when the extracted experience is correct, agents can effectively utilize and transfer it to new tasks (§4.2).

4.1 Errors in Experience Formation

To identify the causes of limited experience transfer, we manually inspect experience accumulated during self-evolution. For each evolution update, we trace how the agent interprets the failed trajectory, how this interpretation informs failure attribution, and how the resulting explanation is consolidated into experience. We analyze Qwen3.5-27B with ReasoningBank (Memory), SkillOpt (Skill), and Evo-Harness (Harness), and GPT-5.6-Sol with Evo-Harness. ReasoningBank is evaluated on ALFWorld and τ^2-Bench-Verified; SkillOpt and Evo-Harness are evaluated on ALFWorld, τ^2-Bench-Verified, and WebShop.

We categorize reasons for incorrect experience formation into four types: **(M1) False-Positive Success Attribution** (False Success), where actions or intermediate states that contributed to failure are credited as successful practices (e.g., treating a budget-wasting exhaustive search as a successful search strategy). **(M2) Misidentified & Hallucinated Failure Attribution** (Misattribution), where relevant causes are overlooked or replaced with unrelated factors or unsupported explanations (e.g., falsely interpreting filters on the wrong category page as evidence that the requested category has been reached). **(M3) Distorted Experience Consolidation** (Distortion), where observations or inferences lose their conditions or acquire unwarranted certainty when converted into experi-

Model	Paradigm	M1	M2	M3	M4
		ALFWorld			
Qwen3.5-27B	Memory	41.3	46.5	32.6	35.5
Qwen3.5-27B	Skill	43.8	39.7	28.8	37.0
Qwen3.5-27B	Harness	29.9	38.6	31.5	36.2
GPT-5.6-Sol	Harness	21.2	31.0	25.7	23.9
		τ^2-Bench-Verified			
Qwen3.5-27B	Memory	34.0	53.2	29.8	42.6
Qwen3.5-27B	Skill	37.8	48.6	35.1	43.2
Qwen3.5-27B	Harness	27.9	44.2	34.9	32.6
GPT-5.6-Sol	Harness	31.7	39.0	19.5	26.8
		WebShop			
Qwen3.5-27B	Skill	43.3	31.3	23.9	38.8
Qwen3.5-27B	Harness	28.2	32.4	23.9	36.6
GPT-5.6-Sol	Harness	19.7	24.6	26.2	23.0

Table 1: Percentage of reviewed updates exhibiting each failure mode. A single update may exhibit multiple failure modes.

ence (e.g., turning "go back before a new search" into "always go back from a results page"). **(M4) Poisonous Future Behavior Guidance** (Harmful Guidance), where incorrect or unsupported judgments become direct instructions that badly conflict with environmental constraints and can lead to further failures (e.g., advising agents to abandon troubleshooting and escalate after three unsuccessful explanations, even when repairs remain available). Some instances may exhibit more than one failure mode. Detailed annotation criteria and cases are provided in Appendix D.2.

Table 1 summarizes the distribution of these errors across models and benchmarks. Misattribution is particularly common in τ^2-Bench Verified, whereas False Success is more prominent in ALFWorld. Stronger models generally exhibit fewer errors, and harness evolution shows lower error rates than memory or skill evolution. These error patterns reveal a weakness in learning under *Cold-Start Failure-Only Evolution*. Unsupported interpretations of failed attempts may be consolidated into reusable experience and carried forward as action rules, potentially guiding subsequent behavior in ways that conflict with task constraints.

4.2 Utilization and Transfer of Correct Experience

Setup. We further investigate whether agents can effectively use failure-derived experience to improve performance on new tasks, even when the experience itself is correct. To this end, we retain only manually verified memory entries or harness modifications at each checkpoint. We evaluate Qwen3.5-27B with ReasoningBank on ALFWorld and τ^2-Bench-Verified, and GPT-5.6-Sol with Evo-Harness on these benchmarks and WebShop. Full experience, verified experience, and no experience are compared on the same source and held-out tasks, with experience fixed during evaluation.

Results & Analysis. Even when the experience extracted from failures is correct, agents still struggle to apply it effectively to held-out tasks. As shown in Figure 4, retaining only verified experience modestly improves source-task recovery in all five configurations, yet final held-out scores remain at or below the no-experience baseline in four.

These results further indicate the challenge of learning transferable experience under *Cold-Start Failure-Only Evolution*. Even when the experience is correct in terms of content, agents still struggle to use it effectively to form transferable capability. Our inspection shows that such experience remains tied to specific steps and conditions of the source task, indicating a gap in extracting generalizable knowledge from harsh and dense failures.

5 Effects of Learning Conditions on Experience Transfer

We next address **RQ3**: what learning conditions are missing for agents to acquire more transferable experience? We systematically relax the CFOE setting along three dimensions by introducing successful experience, exposing verifier feedback, and allowing additional environment interaction during failure reflection.

Setup. We adopt the three self-evolving frameworks with Qwen3.5-27B and evaluate held-out performance on the environments described in §3.1. We first vary the source of trajectories available for experience extraction: **(1) Fail**: learns only from failed interactions in the failure-dense task stream, following the original setting in §3. **(2) Success**: samples tasks that succeeded in all three preliminary rollouts and learns only from success during online evolution. **(3) Mix**: randomly samples tasks and allows learning from both successful and failed interactions during online evolution. Task streams are sampled separately for each setting. We evaluate Success and Mix only with ReasoningBank and SkillOpt, as Evo-Harness retains its failure-focused update procedure. In these comparisons, official verifier judgments remain hidden from the agent and updater, which we denote as **Blind**.

We next isolate the effect of external evaluative signals by exposing the official verifier feedback under failure-only evolution (**Feedback**), including outcome labels, reward scores, and task-specific feedback available in each environment.

Finally, we examine whether additional interaction can compensate for missing information in a failed trajectory. Inspired by the agentic error analysis in Trace2Skill (Ni et al., 2026), we allow the agent to resume interaction with the environment during reflection (**Resume**) to diagnose the failure and explore possible corrections. Reflection is still triggered by the original failed rollout, but the agent may use observations collected during these additional interactions when extracting experience, including those from a subsequently successful attempt. Verifier judgments and ground truth remain hidden. We allocate 10 interaction turns followed

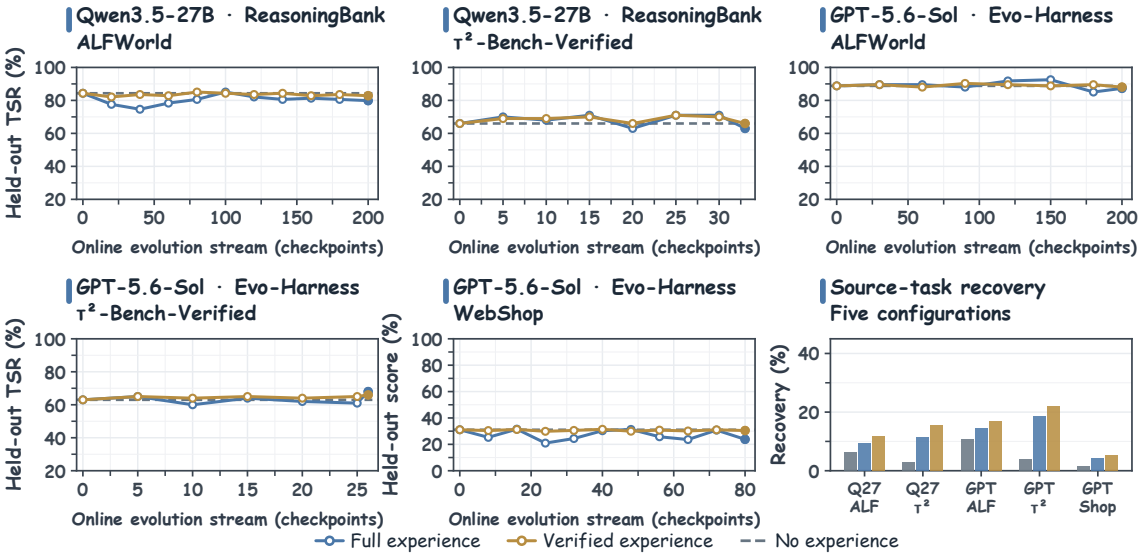


Figure 4: Held-out performance and source-task recovery with full experience, verified experience, and no experience. Dashed lines denote the no-experience held-out baseline.

by one tool-free summary turn in the main comparison. Additional settings and budget comparisons are provided in Appendix D.3.

Results & Analysis. Figure 5 reports held-out performance during online self-evolution under different learning conditions on ALFWorld and τ^2-Bench-Verified. Appendix D.3 provides additional results on WebArena-Verified and WebShop. It further compares 5, 10, and 20 additional interaction turns for Resume, and evaluates verifier feedback under successful and mixed trajectory streams.

Successful interactions support more effective experience transfer. As shown in Figure 5, learning from successful interactions yields larger held-out improvements than learning exclusively from failures. Mix also benefits from incorporating successful interactions, although its gains are generally smaller than those of Success. Successful trajectories provide examples of action sequences that complete a task, giving agents concrete references when extracting reusable experience. These results suggest that access to successful executions is important for learning transferable experience.

Verifier feedback improves failure-only learning, but its benefits remain limited. Providing official verifier judgments improves held-out performance relative to Fail + Blind. The gains nevertheless remain limited compared with those from successful interactions. Feedback supplies additional evidence about task outcomes and unmet requirements, but agents must still infer behavioral corrections and express them as reusable experience. The remaining transfer difficulties suggest that external judgment only partially addresses the challenges of learning from failed interactions.

Interactive failure diagnosis offers further learning opportunities with limited transfer gains. Revisiting failed tasks during reflection also improves held-out performance over the CFOE baseline. Through additional interactions, agents can investigate uncertain steps and explore possible corrections before consolidating experience. These opportunities yield some transferable gains, although the overall improvement remains limited. Together with the feedback results, this finding suggests that additional failure evidence can help learning, while extracting transferable experience remains challenging.

6 Related Work

Experience-Driven Self-Evolving Agents. Recent work explores agents that improve with frozen model parameters by retaining and reusing interaction experience (Wang et al., 2024; Zhang et al., 2026). Experience can be collected offline or accumulated during deployment. Offline, AutoGuide extracts context-aware guidelines from prior trajectories (Fu et al., 2024). Online approaches update environmental rules through interaction (Chen et al., 2024) or maintain evolving memories of strategies and code for subsequent queries (Suzgun et al., 2026). Voyager stores executable skills (Wang et al., 2024), while Agent Workflow

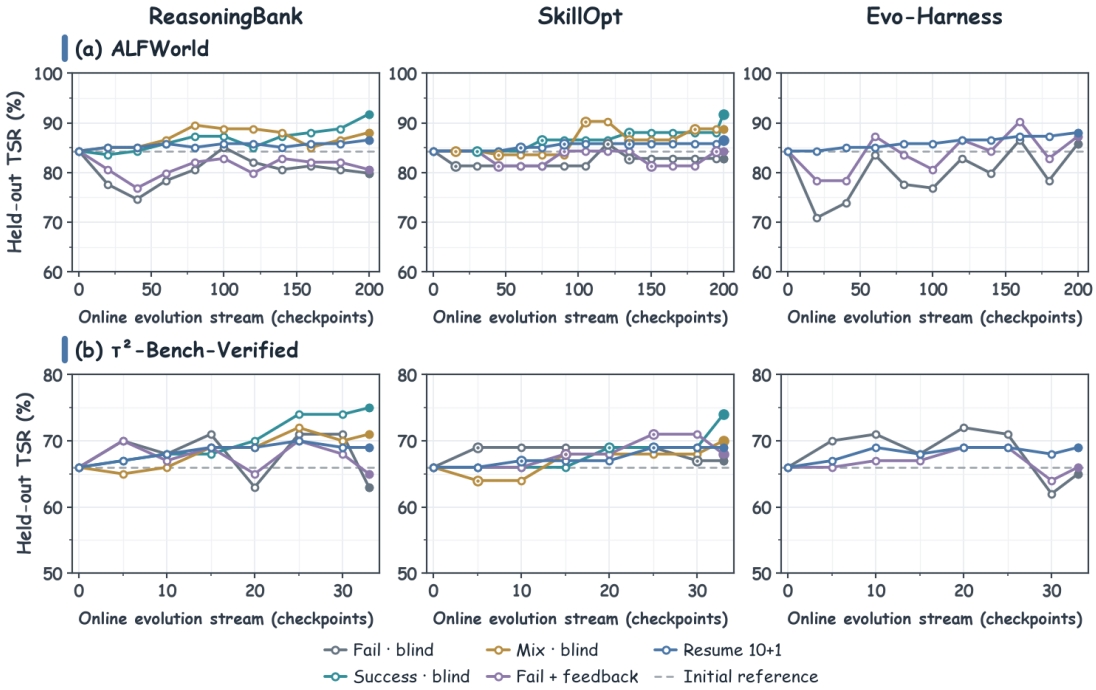


Figure 5: Learning-condition comparisons on ALFWorld and τ^2-Bench-Verified with Qwen3.5-27B. Fail + Blind reproduces the reported failure-only baseline; Resume uses 10 interaction turns and one summary turn. Dashed lines indicate initial held-out performance.

Memory induces reusable workflows (Wang et al., 2025). Recent frameworks distill reasoning memories (Ouyang et al., 2026), refine skill instructions (Yang et al., 2026b), and compile harness guidance (Wei et al., 2026). ACE organizes accumulated strategies into structured playbooks through incremental context updates (Zhang et al., 2026). We examine memory, skill, and harness evolution from a shared cold start as agents learn from failures on initially challenging tasks.

Learning from Failure. Failure learning has been studied through reflection, feedback, and experience reuse. Reflexion stores reflections on task feedback to guide subsequent trials (Shinn et al., 2023), while ExpeL contrasts failed and successful executions to derive cross-task rules (Zhao et al., 2024). Output revision uses model-generated feedback in Self-Refine (Madaan et al., 2023) and tool-based feedback in CRITIC (Gou et al., 2024). For persistent experience updates, ReasoningBank builds memories from self-judged trajectories (Ouyang et al., 2026), and ERL examines failure-derived heuristics and extraction without outcome feedback (Allard et al., 2026). Other approaches check updates against additional evidence. HarnessFix grounds diagnoses in execution traces and harness artifacts and validates proposed repairs (Chen et al., 2026); Sentry retains recovery lessons after checking subsequent interactions and evaluates reuse on unseen tasks without task-level rewards (Ji et al., 2026). These studies connect failure diagnosis to subsequent behavior across feedback conditions. We study failure-dense learning from unsuccessful interactions without verifier explanations, separately evaluating source-task repair and held-out transfer.

7 Conclusion

We study whether self-evolving LLM agents can acquire transferable capability from failure alone through *Cold-Start Failure-Only Evolution* (CFOE). Across multiple self-evolving paradigms, LLM backbones, and interactive environments, we uncover a consistent *repair–generalization gap*: failure-derived experience can repair observed failures but rarely transfers to unseen tasks. Further analyses show that both unreliable failure interpretation and limited transfer of even correct experience contribute to this gap. Successful trajectories provide substantially stronger transfer gains than verifier feedback or additional failure exploration, suggesting that current agents are better at learning *what went wrong* than *what reliably works*.

Limitations

While our study provides a systematic investigation into the limitations of cold-start failure-only evolution in LLM agents, several limitations remain. First, our evaluation prioritizes challenging tasks from a focused set of benchmarks that span both web-based and embodied scenarios. However, these benchmarks may not fully capture the diversity of difficult situations agents encounter in real-world deployment environments, especially those involving multi-agent interactions or multimodal inputs. Extending our analysis to broader distributions of challenging tasks remains an important direction. Second, our experiments study only scale self-evolving agents over a finite number of self-evolution steps, with the longest evaluation covering 800 tasks on ALFWorld. While this already reveals persistent limitations in experience transfer, real-world deployed agents may undergo self-evolution over far longer, and potentially unbounded, time horizons. How experience transfer evolves under such indefinite failure accumulation, and whether transferable gains emerge beyond the studied regime, remain open questions for future work and more scaling exploration.

Overall, this work provides an empirical investigation into the limits of learning from failures in self-evolving agents. We hope future efforts will explore more general, principled, and verifiable mechanisms to support transferable learning from failures in experience-driven AI systems.

Ethical Considerations

This work is conducted solely for research purposes, with the goal of understanding the limitations of learning from failures in self-evolving agents. All experiments are performed in controlled benchmark environments, with restricted learning signals used to study experience formation and transfer. We believe that systematically identifying and characterizing errors in accumulated experience is important for reducing the risk of repeated mistakes that could affect users in real-world applications. By exposing failure modes and transfer limitations in current self-evolving frameworks, this work aims to inform the design of evaluation and oversight mechanisms for more reliable and responsible agentic systems.

References

Marc-Antoine Allard, Arnaud Teinturier, Victor Xing, and Gautier Viaud. 2026. Experiential reflective learning for self-improving LLM agents. In *ICLR 2026 Workshop on Memory for LLM-Based Agentic Systems (MemAgents)*.

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik R Narasimhan. 2026. τ^2-Bench: Evaluating conversational agents in a dual-control environment. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 6736–6782. PMLR.

Jack Bell, Luigi Quarantiello, Eric Nuertey Coleman, Lanpei Li, Malio Li, Mauro Madeddu, Elia Piccoli, and Vincenzo Lomonaco. 2025. The future of continual learning in the era of foundation models: Three key directions. In *Proceedings of the Workshops at the Fourth International Conference on Hybrid Human-Artificial Intelligence*, volume 4074 of *CEUR Workshop Proceedings*, pages 525–548.

Yuxuan Cai, Yipeng Hao, Jie Zhou, Hang Yan, Zhikai Lei, Rui Zheng, Zhenhua Han, Yutao Yang, Junsong Li, Qianjun Pan, Tianyu Huai, Qin Chen, Xin Li, Kai Chen, Bo Zhang, Xipeng Qiu, and Liang He. 2026. Building self-evolving agents via experience-driven lifelong learning: A framework and benchmark. *Preprint*, arXiv:2508.19005. Version 6, revised 26 January 2026.

Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. 2026. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws. *Preprint*, arXiv:2606.06324.

Minghao Chen, Yihang Li, Yanting Yang, Shiyu Yu, Binbin Lin, and Xiaofei He. 2024. AutoManual: Constructing instruction manuals by LLM agents via interactive environmental learning. In *Advances in Neural Information Processing Systems*, volume 37, pages 589–631. Curran Associates, Inc.

Marc-Alexandre Côté, Ákos Kádár, Xingdi Yuan, Ben Kybartas, Tavian Barnes, Emery Fine, James Moore, Matthew Hausknecht, Layla El Asri, Mahmoud Adada, Wendy Tay, and Adam Trischler. 2019. TextWorld: A learning environment for text-based games. In *Computer Games*, volume 1017 of *Communications in Computer and Information Science*, pages 41–75, Cham. Springer.

Alejandro Cuadron, Pengfei Yu, Yang Liu, and Arpit Gupta. 2026. SABER: Small actions, big errors—safeguarding mutating steps in LLM agents. In *ICLR 2026 Workshop on Memory for LLM-Based Agentic Systems (MemAgents)*.

DeepSeek-AI. 2026. DeepSeek-V4: Towards highly efficient million-token context intelligence. *Preprint*, arXiv:2606.19348.

Shihan Dou, Ming Zhang, Chenhao Huang, Jiayi Chen, Feng Chen, Shichun Liu, Yan Liu, Chenxiao Liu, Cheng Zhong, Zongzhang Zhang, Tao Gui, Chao Xin, Chengzhi Wei, Lin Yan, Qi Zhang, and Xuanjing Huang. 2025. EvaLearn: Quantifying the learning capability and efficiency of LLMs via sequential problem solving. In *Advances in Neural Information Processing Systems*, volume 38, pages 125892–125945. Curran Associates, Inc.

Amine El Hattami, Megh Thakkar, Nicolas Chapados, and Christopher Pal. 2025. WebArena Verified: Reliable evaluation for web agents. In *Workshop on Scaling Environments for Agents*.

Yao Fu, Dong-Ki Kim, Jaekyeom Kim, Sungryull Sohn, Lajanugen Logeswaran, Kyunghoon Bae, and Honglak Lee. 2024. AutoGuide: Automated generation and selection of context-aware guidelines for large language model agents. In *Advances in Neural Information Processing Systems*, volume 37, pages 119919–119948. Curran Associates, Inc.

Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Qihan Ren, Yiran Wu, Hongru Wang, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, and 8 others. 2026. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *Transactions on Machine Learning Research*.

Gemma Team. 2026. Gemma 4 technical report. *Preprint*, arXiv:2607.02770.

GLM-5-Team, Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, Chenzheng Zhu, Congfeng Yin, Cunxiang Wang, Gengzheng Pan, Hao Zeng, Haoke Zhang, Haoran Wang, Huilong Chen, and 167 others. 2026. GLM-5: from vibe coding to agentic engineering. *Preprint*, arXiv:2602.15763.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*, volume 2024, pages 57734–57811.

Changxiu Ji, Amy Lu, Qizheng Zhang, and Kunle Olukotun. 2026. Sentry: Learning to recover from LLM agent failures at test time. *Preprint*, arXiv:2610.02994.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, pages 611–626.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, pages 46534–46594. Curran Associates, Inc.

Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjun Jiang. 2026. Trace2Skill: Distill trajectory-local lessons into transferable agent skills. *Preprint*, arXiv:2603.25158. Accepted to EMNLP 2026 (main conference).

OpenAI. 2026a. GPT-5.6: Frontier intelligence that scales with your ambition. Accessed: 2026-10-13.

OpenAI. 2026b. Introducing GPT-5.5. Accessed: 2026-10-13.

Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. 2026. ReasoningBank: Scaling agent self-evolving with reasoning memory. In *International Conference on Learning Representations*, volume 2026, pages 94327–94354.

Qwen Team. 2026. Qwen3.5: Towards native multimodal agents.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652. Curran Associates, Inc.

Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer, and Dieter Fox. 2020. ALFRED: A benchmark for interpreting grounded instructions for everyday tasks. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10737–10746.

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2021. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*.

David Silver and Richard S. Sutton. 2025. Welcome to the era of experience. Preprint of a chapter for Designing an Intelligence.

Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. 2026. Dynamic cheatsheet: Test-time learning with adaptive memory. In *Proceedings of the 19th Conference of the European*

Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), pages 7080–7106. Association for Computational Linguistics.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 63897–63911. PMLR.

Tianxin Wei, Zhan Shi, Minhua Lin, Bing He, Zewen Liu, Yisi Sang, Yuanchen Bei, Xuying Ning, Jiaru Zou, Ting-Wei Li, Xiao Lin, Yanjun Zhao, Chi Wang, Benoit Dumoulin, Dakuo Wang, Jingrui He, and Hanqing Lu. 2026. Evo-Harness: Context-to-harness skill compilation for self-evolving agents. *arXiv preprint arXiv:2608.15071*. Accepted to EMNLP 2026 (main conference).

Shidong Yang, Ziyu Ma, Tongwen Huang, Xucong Wang, Renda Li, Yiming Hu, Yong Wang, and Xiangxiang Chu. 2026a. SkillForge: Evolving verifiable skills for reinforcement learning agents. *Preprint*, arXiv:2608.24747.

Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. 2026b. SkillOpt: Executive strategy for self-evolving agent skills. *arXiv preprint arXiv:2605.23904*.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. WebShop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems*, volume 35, pages 20744–20757. Curran Associates, Inc.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*, volume 2026, pages 86069–86100.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17):19632–19642.

A Self-Evolving Agents

We present detailed overviews of the three self-evolving agent frameworks used in our experiments: ReasoningBank, SkillOpt, and Evo-Harness. These frameworks represent three distinct self-evolution paradigms, evolving a reasoning memory repository, agent skills, and runtime harness, respectively, while keeping the underlying model weights fixed. We introduce the core design and update mechanism of each framework below.

- **ReasoningBank** exemplifies a memory-evolution paradigm for reasoning memory consolidation. It maintains a continuously growing memory that stores distilled reasoning patterns extracted from the agent's own interaction history, including both successful and failed attempts. After each task execution, ReasoningBank reflects on the interaction trajectory and distills the experience into higher-level memory items intended for reuse across tasks. For each new task, relevant memory items are retrieved and incorporated into the model context to guide subsequent reasoning and actions. Under *Cold-Start Failure-Only Evolution*, all trajectories admitted to evolution are harsh failures. ReasoningBank therefore has to extract experience exclusively from failed interactions, with no end-to-end successful trajectories that directly demonstrate effective workflows or provide success–failure comparison.

- **SkillOpt** adopts a skill-evolution paradigm in which a natural-language skill document serves as the trainable external artifact for a frozen execution agent. During training, SkillOpt iteratively refines its skill document across successive rollout batches. After each batch, it organizes successful and failed trajectories into separate reflection minibatches. The optimizer reflects on these minibatches to propose structured edits to the current skill. The proposed edits are then aggregated and ranked, with selected edits applied within a fixed textual edit budget to produce a candidate skill. SkillOpt then evaluates the candidate skill on a held-out validation split, separate from the test set, and adopts it only if it outperforms the current skill. Rejected edits and their validation outcomes are retained to inform subsequent optimization, while slow updates and meta-skill memory provide guidance across epochs. Under *Cold-Start Failure-Only Evolution*, only harsh failures are accessible, so SkillOpt revises its skill using reflection minibatches drawn exclusively from failed trajectories.

- **Evo-Harness** evolves the harness of a frozen base agent through context-to-harness skill compilation, converting execution experience into reusable guidance. The resulting harness combines general strategies applicable across tasks with specialized procedures for recurring task types, interfaces, and domains. Before each task, Evo-Harness retrieves relevant guidance from the current harness and incorporates it into the agent's context to support planning, tool use, verification, and recovery. It then examines failed or negatively evaluated trajectories to extract candidate lessons, recording the supporting evidence and conditions under which each lesson may apply. After each batch, an evolver reviews these lessons against the existing harness and decides which to incorporate, merge, or discard. Lessons associated with recurring task types, interfaces, or domains are integrated into the corresponding topic-specific procedures, while patterns shared across tasks are used to update the general strategies. The updated harness then guides subsequent tasks, allowing both general and topic-specific guidance to evolve as experience accumulates. Under *Cold-Start Failure-Only Evolution*, we follow this evolution process but restrict all experience updates to harsh failure trajectories.

B Environments and Benchmarks

Our experiments span diverse agentic settings, including household embodied task execution, long-horizon interactive tool use, online shopping, and complex web navigation/assistance. All environments guarantee multi-turn, long-horizon interaction rather than single-shot decisions; for instance, **τ^2-Bench-Verified** allows trajectories of up to 200 steps, highlighting long-horizon planning and recovery. However, because our method surrounds harsh failures, we pay extra attention to choose environments whose task instructions and infrastructure are community verified or otherwise guaranteed to be solvable (e.g., ALFWorld goals aligned with expert-validated ALFRED tasks,

human-authored WebShop requests, and the verified releases of τ^2-Bench and WebArena). This choice helps ensure that observed failures are mainly attributable to task difficulty and agent capability limitations, rather than tasks being inherently unsatisfiable or requiring unreachable environment states. In the following, we describe each environment and the data overview used in our study.

B.1 Household Embodied Environment

ALFWorld (Shridhar et al., 2021) provides text-based household environments aligned with the embodied tasks in ALFRED (Shridhar et al., 2020). Given a task instruction, the agent moves to explore the environment, locate objects, and manipulate them through text commands. Completing these tasks requires the agent to track the locations and states of the objects and perform actions in the order required by the task instruction. We use the TextWorld interface (Côté et al., 2019) and combine the 3,553 training tasks with the 140 seen-validation tasks to form a candidate pool of 3,693 tasks. All 134 unseen-validation tasks are reserved for held-out evaluation.

B.2 Long-horizon Interactive Tool Use

τ^2-Bench-Verified (Barres et al., 2026; Cuadron et al., 2026) is an interactive tool-use benchmark that spans the airlines, retail, and telecom domains. Agents gather information through multi-turn dialog with a simulated user, reason over domain policies, and execute tool calls to complete requests. These calls allow the agent to inspect records and modify the environment state, with later actions depending on information and results obtained earlier in the interaction. The verified release corrects inconsistencies among task instructions, expected actions, policies, and database contents in the original benchmark. In our experiments, we use GPT-5.5 (OpenAI, 2026b) as the default user simulator. Our candidate pool comprises 178 training tasks across the three domains, and the 100 test tasks form the holdout set. We use the benchmark's official evaluator and count a task as successful only when it receives the full reward of 1 and ends with a valid conversation termination.

B.3 Web Environment

WebArena-Verified (El Hattami et al., 2025) provides a curated set of tasks on functional websites, with deterministic evaluators that check agent responses and recorded browser network activity. We use version 1.2.3 and evaluate four sites separately: Shopping, Shopping Admin, Reddit, and GitLab. These sites cover online shopping, shopping administration, social forum, and collaborative software development. Agents follow natural-language instructions by navigating pages, finding information, and performing requested operations through the browser. Within each site, we sampled approximately one quarter of the tasks across templates for held-out evaluation and use the remaining tasks as candidates for evolution-stream construction. The set of held-outs contains 47 Shopping, 46 Shopping Admin, 27 Reddit, and 45 GitLab tasks; their corresponding candidate pools contain 140, 136, 79, and 135 tasks. A task is successful when all checks required by the official verified-version evaluator pass.

WebShop (Yao et al., 2022) is a simulated e-commerce environment with a catalog of 1.18 million real-world products and 12,087 human-written shopping instructions. Each task requires an agent to find and purchase an item that satisfies the requested attributes, options, and budget. The agent must interpret the request, formulate search queries, and extract relevant information from product pages to guide its choices. Our experiments use a fixed set of 500 tasks that span beauty, electronics, fashion, gardening, and grocery products. We reserve 80 tasks through category-stratified sampling for held-out evaluation and treat the remaining 420 as the candidate pool for evolution. Evaluation reports the mean task reward on a 0–1 scale. A reward of 1 denotes full matches.

C Implementation Details

C.1 Execution Details

We provide additional implementation details for maximum step budgets set by each environment and how we adapt SkillOpt from its original offline paradigm to online self-evolution setting in this work. All frameworks use the same underlying model for both rollout and reflection.

Rollout settings. We allow up to 30 rollout steps for ALFWorld and WebArena-Verified, 200 steps for τ^2-Bench-Verified, and 20 steps for WebShop. The allocated step budgets provide sufficient chance for agents to complete the corresponding tasks. The four benchmarks have also been rigorously reviewed by previous researchers. ALFWorld provides expert trajectories demonstrating that its

tasks are solvable, while WebShop uses human-written shopping instructions that have been manually reviewed. For τ^2-Bench and WebArena, we adopt their verified releases to support more reliable and consistent evaluation.

SkillOpt Adaptation. We adapt SkillOpt from the official *offline* paradigm to an *online* self-evolution setting. The online task stream can be traversed once rather than for multiple epochs. After each batch of task executions, SkillOpt organizes trajectories into reflection minibatches and proposes suggestive skill edits. The candidate skill is then evaluated using SkillOpt's original validation gate, while the held-out set is reserved exclusively for checkpoint evaluation. Under the blind setting, validation scores are used only by the controller to accept or reject candidate skills; neither the execution nor the reflection agent receives these scores or verifier feedback. Blindness thus concerns the information available to these agents, with score-based selection retained at the controller level.

C.2 Failure-Dense Online Evolution Streams Construction.

For each benchmark, we construct a backbone-specific online task stream by selecting tasks that consistently challenge the backbone from the candidate pool described in Appendix B. We run a baseline react agent (Yao et al., 2023) (without external experience) on each candidate task up to three independent attempts, and keep tasks that fail all runs, with infrastructure and evaluator errors strictly excluded. This process filters out tasks that can be luckily solved by chance on a retry and leaves tasks that are consistently difficult for the base agent. We then sample without replacement to balance the final stream across task type (ALFWorld), domain (τ^2-Bench-Verified), and product category (WebShop). For WebArena-Verified, we construct a separate stream for each site. We target 200 tasks for ALFWorld, 50 for τ^2-Bench-Verified, 80 for WebShop, and 30 per WebArena-Verified site. If fewer tasks qualify 3 failures, we include all of them as failure-dense online evolution stream.

C.3 Verifier Feedback Blind

Table 2 presents representative examples of the verifier feedback withheld under the blind setting described in Section 3.1. These examples include outcome labels, reward scores, and detailed checks of task completion. The agent retains access to interaction observations, including action outcomes, state changes, and execution errors, which provide the local evidence for experience extraction.

Table 2: Representative examples of verifier feedback withheld from the agent across the four environments.

ALFWorld

=== Outcome ===
success=False score=0.0 steps=30 terminated=False truncated=True

=== Grader Feedback ===
Official task success=False reward=0.
Official goal conditions (checked independently under the goal's static constraints; separate conditions need not share the same object):
- condition 3: istoggled: not satisfied
- condition 5: atlocation: satisfied
- condition 6: inreceptacle: satisfied
- condition 8: holds: not satisfied

Final status: Failed
Final score: 0.0

τ^2-Bench-Verified

=== Outcome ===
success=False score=0.0 steps=10 terminated=True truncated=False

=== Grader Feedback ===
Official tau2 evaluator verdict: FAILED
- database check FAILED: the final environment state does not match the expected state
- expected action MISSING/WRONG: `modify_pending_order_address({'order_id': '#W1603792', 'address1': '592 Elm Avenue', 'address2': 'Suite 978', 'city': 'Houston', 'country': 'USA', 'state': 'TX', 'zip': '77242'})`
- expected action MISSING/WRONG: `modify_user_address({'user_id': 'sophia_martin_8570', 'address1': '592 Elm Avenue', 'address2': 'Suite 978', 'city': 'Houston', 'country': 'USA', 'state': 'TX', 'zip': '77242'})`
- expected action MISSING/WRONG: `modify_pending_order_items({'order_id': '#W1603792', 'item_ids': ['6501071631'], 'new_item_ids': ['2106335193'], 'payment_method_id': 'credit_card_5694100'})`

Additional official checks:
Official reward basis: DB, COMMUNICATE
Official reward breakdown: DB=0.0, COMMUNICATE=1.0

Final status: Failed
Final score: 0.0

WebArena-Verified

=== Outcome ===
success=False score=0.0 steps=30 terminated=False truncated=True

=== Grader Feedback ===
official_status=failure binary_score=0.0
Requirement 1 (final response requirement): not fulfilled
- response task_type: expected=MUTATE actual=NAVIGATE
- response status: expected=SUCCESS actual=UNKNOWN_ERROR
Requirement 2 (navigation or state-change requirement): not fulfilled
- required navigation or state-change evidence is missing
Additional official checks:
Requirement 1 official score=0
Requirement 2 official score=0
Requirement 2 check missing_navigation_event: failure

Final status: Failed
Final score: 0.0

WebShop

=== Outcome ===
success=False score=0.4 steps=9 terminated=True truncated=False

=== Grader Feedback ===
Official purchase reward=0.4.
product_type=1 (fulfilled)
attributes=0.5 (not fully fulfilled)
options=0 (not fully fulfilled)
price=1 (fulfilled)
Official type check query_match=False
Official type check category_match=True
WebShop has a large product catalog, not a single unique correct product. Purchases receive a score from 0 to 1 according to how well they match the user's requirements; aim for a high matching score. A positive score confirms that a purchase was completed; completing a purchase is a prerequisite for earning any score. Partial credit does not mean every requirement was fulfilled. Use the reported component scores to identify supported matches and remaining gaps, without assuming an unreported component passed.

Final status: Failed
Final score: 0.4

C.4 Held-Out Evaluation

For each benchmark, we reserve a fixed held-out set before constructing the online evolution stream, as described in Appendix B. The agent is evaluated on the same tasks at successive checkpoints, including the initial state and the final checkpoint. Each task is executed once from its initial state under the benchmark's execution budget. The checkpoint's complete memory, skill, or harness remains fixed across all held-out tasks. Reflection and experience updates are disabled, and held-out trajectories and evaluation feedback are not used in subsequent online updates. For SkillOpt, the held-out set is separate from the validation set used to accept or reject candidate skills.

Performance is measured by task success rate on ALFWorld, τ^2-Bench-Verified, and WebArena-Verified, and by mean official score on WebShop. Each checkpoint contributes one point to its configuration's held-out curve; the curve includes both the initial state and the final checkpoint.

C.5 Source-Task Recovery Evaluation

The original online execution establishes that each source task failed before its trajectory was used for an update. For each effective update, we freeze the complete experience state immediately after the update and give every valid source task one further attempt from its initial state, using the original execution budget. ReasoningBank is evaluated after each memory update, with source-task memories eligible for its native retrieval. SkillOpt includes only accepted proposals, and Evo-Harness includes only batches that change the harness content. Reflection, experience updates, and validation are disabled during recovery evaluation, and recovery results are not returned to the updater.

Each configuration produces a single recovery score by aggregating these attempts over the entire online run. For ALFWorld, τ^2-Bench-Verified, and WebArena-Verified, this is the total number of successful recovery attempts divided by the total number of valid recovery attempts. For WebShop, we average official scores over those attempts. Each valid recovery attempt contributes equally to the aggregate. The evaluated source sets depend on each framework's online trajectories and update decisions. Invalid or incomplete evaluations are reported separately and are not counted as task failures or zero scores.

D Additional Experimental Results

D.1 Results Across Models and Benchmarks

Figure 6 presents ReasoningBank results on the four WebArena-Verified sites and WebShop results for SkillOpt and Evo-Harness, with eight backbones in each panel. As in the main figures, each configuration is represented by a held-out curve and a recovery bar. WebArena-Verified reports task success rates; WebShop reports mean official scores for both held-out evaluation and recovery. Recovery bars aggregate attempts over the entire online run.

WebArena-Verified: Limited transfer across sites. Across the four sites, most configurations finish at or below their initial held-out TSR. For example, Qwen3.5-4B declines from 25.93% to 7.41% on Reddit, while DeepSeek-V4-Pro remains at 47.83% on Shopping Admin. Recovery rates range from 3.5% to 15.4%, indicating that some original failures can be recovered even when held-out performance does not improve. Qwen3.5-35B-A3B, for instance, recovers 11.1% of source tasks on Reddit, while its held-out TSR falls from 55.56% to 48.15%. These results extend the main finding to web navigation: partial recovery of source tasks does not consistently translate into improvements on new tasks.

WebShop: Held-out gains remain limited across configurations. Most configurations show little change or lower held-out scores after self-evolution. GPT-5.6-Sol with Evo-Harness declines from 31.06% to 23.79%, while GLM-5.1 with SkillOpt changes only from 27.40% to 27.70%. Some SkillOpt configurations improve, including Qwen3.5-27B from 21.64% to 32.18%, but these gains do not extend across the evaluated backbones and frameworks. Recovery scores range from 2.5% to 11.8% among configurations with valid recovery evaluations. Together, the held-out and recovery results indicate limited benefits from *Cold-Start Failure-Only Evolution* in online shopping, with gains concentrated in a subset of configurations.

D.2 Annotation Protocol for Experience-Formation Case Study

Annotators. We hired three annotators to conduct the manual inspection and labeling. All annotators are young adults with higher-education backgrounds (i.e., currently enrolled in or graduated

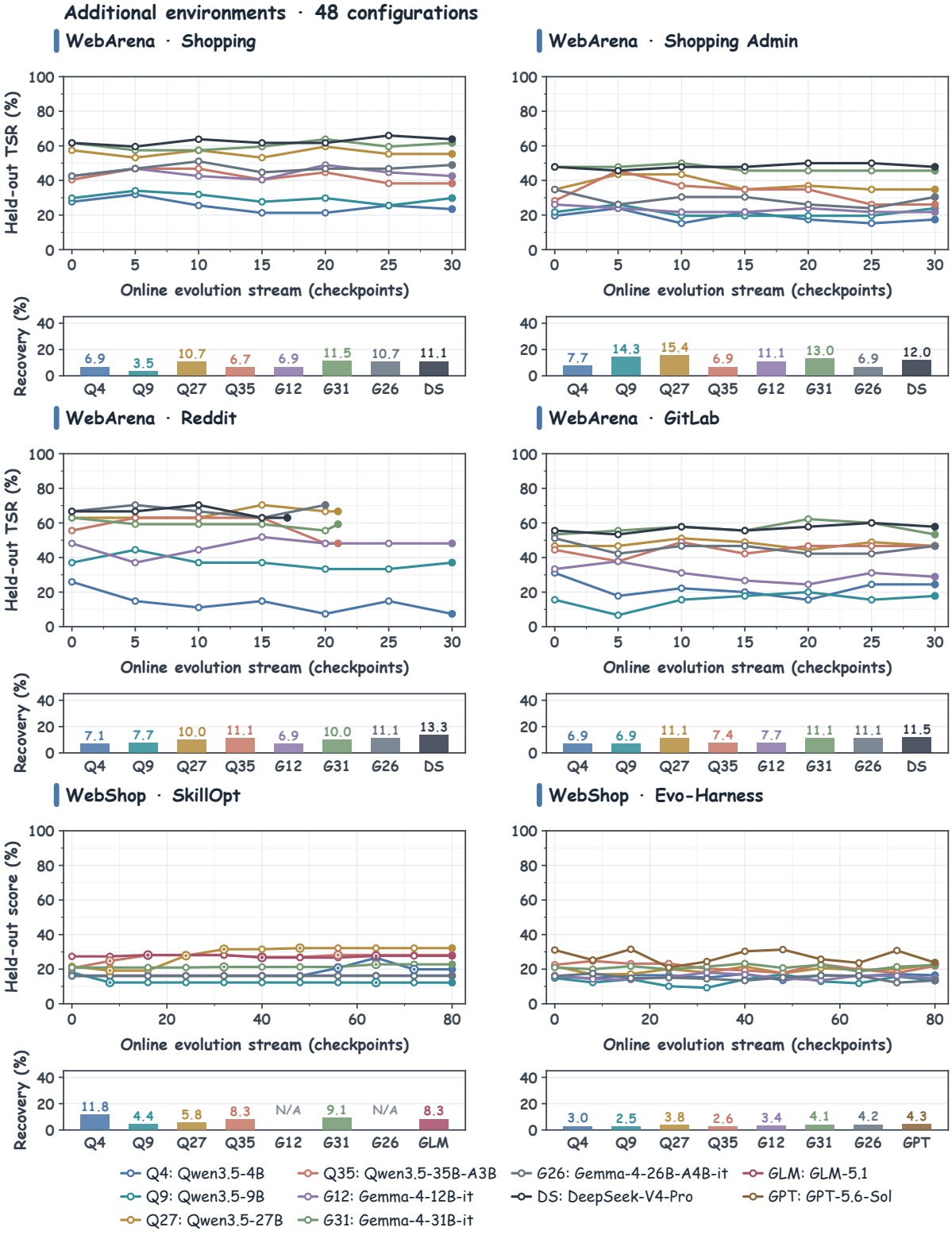


Figure 6: Held-out performance and source-task recovery during cold-start failure-only evolution on WebArena-Verified (ReasoningBank) and WebShop (SkillOpt and Evo-Harness). WebArena-Verified reports task success rates; WebShop reports mean official scores. N/A indicates that no SkillOpt candidate update passed validation, leaving no accepted updates for recovery evaluation.

from a university program). They were financially compensated following a pre-agreed hourly rate.

Annotation scope and unit. We focus on experience updates produced during CFOE. Each annotation instance consists of: (i) the original task and context, (ii) the failed trajectory or trajectories used for the update, (iii) the corresponding reflection, and (iv) the resulting memory entry, skill edit, or harness modification, with the relevant content before and after an edit. Annotators record each failure mode supported by the evidence, allowing multiple modes to co-occur in the same instance.

Label set and detailed criteria. We categorize reasons for incorrect experience formation into the following four types:

- False-Positive Success Attribution, where actions, procedures, or intermediate states that contributed to failure are credited as effective steps and retained as successful practices (e.g., an exhaustive search that consumes the interaction budget is recorded as a successful search strategy). Detailed examples are shown in Table 3.
- Misidentified & Hallucinated Failure Attribution, where the reflection overlooks relevant failure causes, attributes failure to unrelated factors, or introduces explanations unsupported by the trajectory and environment (e.g., falsely treating filters on the wrong category page as confirmation that the requested category has been reached, leaving the actual category mismatch undiagnosed). Detailed examples are shown in Table 4.
- Distorted Experience Consolidation, where observations or inferences lose necessary conditions, acquire broader applicability, or become more certain when converted into reusable experience (e.g., the requirement to go back before issuing a new search becomes a rule to always go back from a results page). Detailed examples are shown in Table 5.
- Poisonous Future Behavior Guidance, where incorrect or unsupported judgments are converted into actionable rules that can induce further failures when reused (e.g., instructing agents to stop troubleshooting and escalate after three unsuccessful explanations, even when executable repairs remain; subsequent agents follow this rule and leave those repairs unattempted). Detailed examples are shown in Table 6.

D.3 Effects of Learning Conditions

We provide the settings and additional comparisons for §5. All conditions use Qwen3.5-27B. The failure-only comparison covers 12 framework–benchmark configurations: all three frameworks on ALFWorld and τ^2-Bench-Verified, ReasoningBank on four WebArena-Verified sites, and SkillOpt and Evo-Harness on WebShop. Success and Mix cover the nine ReasoningBank and SkillOpt configurations.

Learning material and feedback. Fail + Blind reuses the CFOE results reported in Figures 2 and 6. Success selects tasks that succeeded in all three preliminary rollouts and admits only successful online interactions to learning. Mix uses randomly sampled tasks and admits both outcomes, without imposing a fixed success-to-failure ratio. Feedback exposes the official verifier judgments illustrated in Table 2, including expected actions and their arguments where available. No expert trajectories or complete reference solutions are provided. Figure 7 extends the main comparison to web navigation and online shopping.

Interactive diagnosis budget. We compare 5, 10, and 20 additional interaction turns, each followed by one tool-free summary turn (Figure 8). The interaction budget applies to diagnosis during reflection and is separate from the original task-execution budget. The original rollout remains recorded as a failure regardless of subsequent attempts. Reflection can use all evidence gathered during diagnosis, while official verifier judgments and ground truth remain hidden. This comparison examines whether more opportunities to investigate a failure yield experience that transfers to unseen tasks.

Feedback with successful and mixed interactions. Figure 9 includes only Success + Feedback and Mix + Feedback, with initial performance as a reference. The corresponding blind conditions appear in Figures 5 and 7. This comparison examines the usefulness of verifier judgments when successful executions are already available for learning.

Evaluation and interpretation. We evaluate held-out performance with experience fixed, following Appendix C.4. These comparisons concern

transfer to unseen tasks; successful attempts during diagnosis are not counted as source-task recovery results. Different conditions need not share task streams or update schedules. For SkillOpt, inherited checkpoints retain the accepted skill, and an accepted update is distinct from an evaluation checkpoint.

Successful interactions improve transfer in additional environments. Figure 7 extends the main comparison to web navigation and online shopping. Success yields the highest final held-out performance on the four WebArena-Verified sites and with SkillOpt on WebShop. Mix also improves over the initial reference, although its final performance remains below Success. The failure-based conditions show more varied benefits: Feedback and Resume improve on Fail + Blind in several configurations, while their final scores overlap with other conditions in some settings. These results extend the main observation that successful interactions provide useful material for transferable experience, while additional judgment or exploration offers more limited benefits when learning starts from failures.

Increasing the diagnosis budget yields limited additional gains. Figure 8 compares 5, 10, and 20 interaction turns, each followed by one tool-free summary turn. Allowing interactive diagnosis improves final held-out performance over Fail + Blind, and larger budgets generally produce higher final scores. The differences between budgets are modest, with some configurations reaching the same final score under two budgets. For example, ReasoningBank on Reddit finishes at the same level with 5 and 10 interaction turns despite differences during learning. Additional opportunities to investigate a failure therefore help, but the small gains from extending the budget suggest that more interaction alone only partially addresses the difficulty of extracting transferable experience.

Feedback also benefits learning with successful and mixed interactions. Figure 9 reports Success + Feedback and Mix + Feedback, both of which improve over initial held-out performance. Compared with their blind counterparts in Figures 5 and 7, feedback provides additional gains in several configurations, particularly for Mix. These gains vary across environments; Success, for example, reaches the same final score with and without feedback on several WebArena-Verified sites. This comparison suggests that verifier judgments can complement successful examples, while the benefits depend on the learning material already available. Together with the failure-only comparison, the results indicate that the usefulness of external judgment depends on the interactions from which agents extract experience.

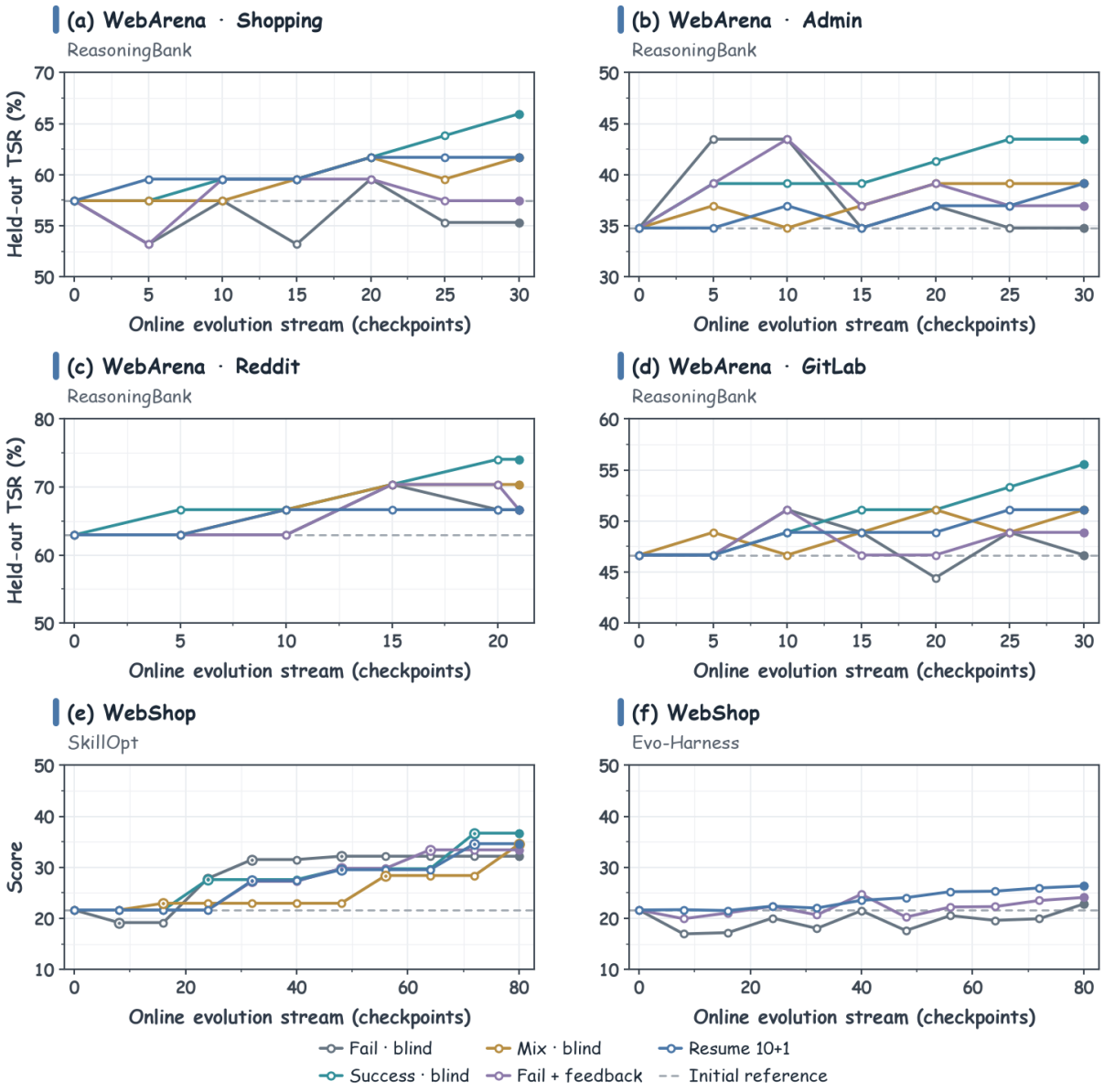


Figure 7: Learning-condition comparisons on WebArena-Verified and WebShop with Qwen3.5-27B, extending Figure 5. Resume uses 10 interaction turns and one summary turn.

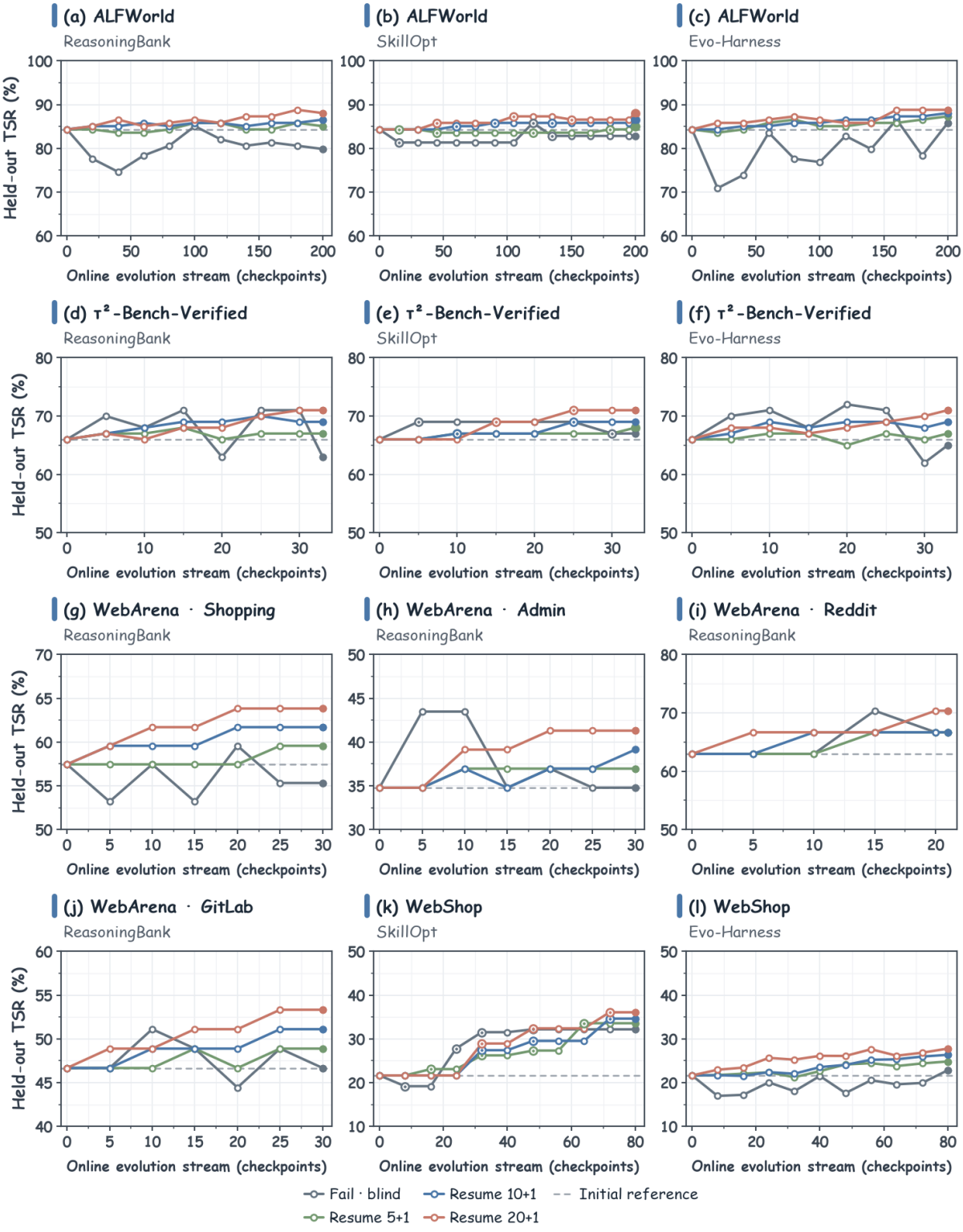


Figure 8: Resume budgets across all 12 configurations with Qwen3.5-27B. Each budget allows 5, 10, or 20 interaction turns followed by one tool-free summary turn.

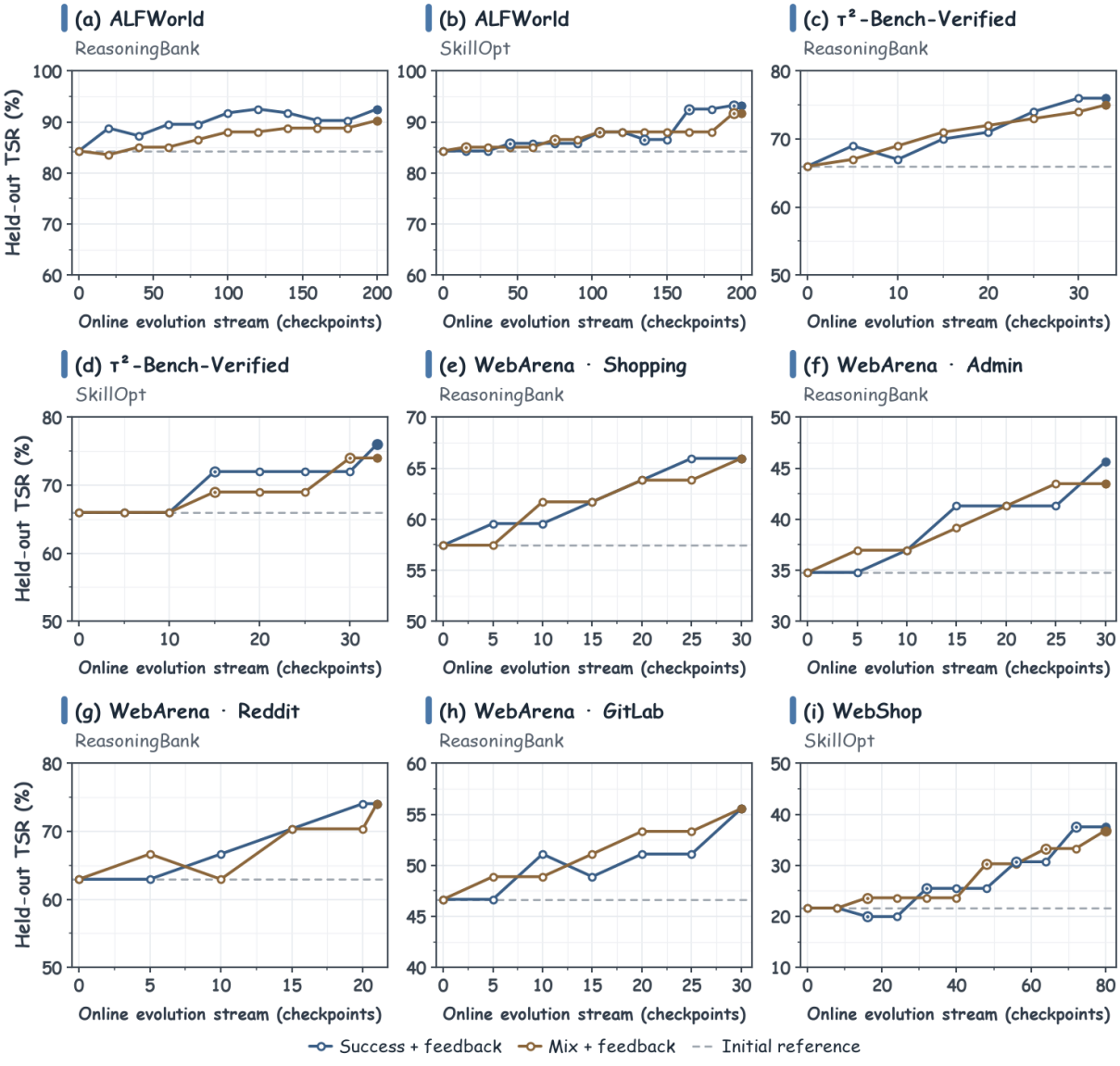


Figure 9: Success + Feedback and Mix + Feedback across nine ReasoningBank and SkillOpt configurations with Qwen3.5-27B. Dashed lines indicate initial held-out performance.

D.4 Experience-Formation Cases

Tables 3–6 provide the cases accompanying the annotation criteria in Appendix D.2.

Table 3: Representative example of False-Positive Success Attribution.

Task Instruction

Your task is to: look at statue under the desklamp.

Trajectory Summary

Step 1.
<thought> I will start with drawer 1. If not, I will proceed sequentially.
<action> go to drawer 1
Step 21.
<thought> I have finished checking all drawers (1 through 10). [...] I should start checking the shelves systematically.
<action> go to shelf 1
Step 22.
<observation> You arrive at shelf 1. On the shelf 1, you see a alarmclock 1, and a statue 1.
<thought> I have found the statue 1 on shelf 1. [...] I should continue searching the shelves for the desklamp.
<action> go to shelf 2
Step 30.
<observation> You arrive at shelf 9. On the shelf 9, you see nothing.
<action> go to bed 1

Generated Experience

Memory Item 1 – Systematic Exhaustion of Container Sets.
Description: When a target object is not immediately visible, agents should exhaustively search all containers of a specific type in a fixed numerical order before switching to a different container type.
Content: The agent successfully located the target statue by first iterating through all 10 drawers (1–10) and then all 9 shelves (1–9) in strict numerical sequence, ensuring no potential location within those categories was skipped. This ordered approach prevents premature switching between container types (e.g., jumping from drawers to shelves) which could lead to missed items or redundant navigation. Future agents should maintain this "finish one category before starting another" discipline when the environment lists multiple instances of similar containers.

Analysis

Actual failure. Steps 1–20 exhausted all drawers. After finding statue 1 at Step 21, the agent left it and spent the remaining steps on shelves and the bed, never completing the statue–desklamp interaction. Hidden environment outcome: success=False, score=0.0, steps=30, terminated=False, truncated=True.
Experience error. Memory Item 1 calls the exhaustive search successful, although the statue appeared at the first shelf and the drawer search was unnecessary. It turns budget-wasting failure behavior into a reusable success rule.

Table 4: Representative example of Misidentified and Hallucinated Failure Attribution.

Task Instruction

Open the Cabinets, Racks & Shelves category page to browse products

Trajectory Summary

Step 1.
<thought> The task is to open the “Cabinets, Racks & Shelves” category page. [...] “Home & Kitchen” [...] or “Tools & Home Improvement” [...] seem like the most likely parent categories. Let me start by clicking on “Home & Kitchen” to explore its subcategories.
<action> click(’473’)
Step 2.
<thought> I’m now on the Home & Kitchen category page. [...] It might be under “Storage & Organization” [...] or “Furniture”. Let me click on “Storage & Organization” first as cabinets and racks are typically storage solutions.
<action> click(’1569’)
Step 4.
<thought> I’ve navigated to the “Racks, Shelves & Drawers” category page (cat=177). The task asks for “Cabinets, Racks & Shelves” category. [...] The “Now Shopping by” section shows I’m currently in “Racks, Shelves & Drawers” category. This appears to be the closest match to the requested “Cabinets, Racks & Shelves” category. [...] The current page seems to be the target category page. Let me submit the response indicating success.
<action> submit_response({“task_type”: “NAVIGATE”, “status”: “SUCCESS”, “retrieved_data”: null, “error_details”: null})

Generated Experience

Memory Item 2 – Verifying Category Context via Breadcrumbs and Filters.
Description: Procedure for confirming the current location in a site hierarchy using page elements other than the main navigation.
Content: After navigating deep into a category tree, confirm the current context by inspecting “Now Shopping by” sections, breadcrumbs, or active filter labels rather than relying solely on the page title or URL. If the page displays a “Remove This Item” link next to the current category name or shows price filters specific to that category, this serves as strong evidence that the agent has successfully entered the intended product scope.

Analysis

Actual failure. The agent opened Home & Kitchen > Storage & Organization > Racks, Shelves & Drawers and knowingly accepted it as a close match. The evaluator required Office Products > Cabinets, Racks & Shelves, an exact category with a different name and parent path.
Hidden environment outcome: success=False, score=0.0, steps=4, terminated=True, truncated=False.
Experience error. The trajectory already used “Now Shopping by” and correctly observed the current category. Memory Item 2 repeats that check but misses the decisive entity mismatch. Breadcrumbs and filters prove which page is open; they do not prove that a semantically similar page is the requested page. The reflection therefore turns evidence of the wrong category into evidence of success instead of requiring exact name and parent-path matching.

Table 5: Representative example of Distorted Experience Consolidation.

Task Instruction

Find me 1080p hd video surveillance for motion detection, and price lower than 50.00 dollars

Trajectory Summary

System instruction. On a search-results page you MUST click [Back to Search] before issuing a new search[...]; otherwise you will get "Invalid action!".
Step 5.
<action> search[security camera 1080p under 50]
<observation> [...] [Back to Search] [...] Page 1 (Total results: 50) [...]
Step 6.
<action> search[wifi security camera under 50]
<observation> Invalid action!
Step 7.
<action> click[Back to Search]
<observation> [...] [Search]

Generated Experience

Reflection reasoning.
A critical observation is the presence of a specific rule in the system prompt: *On a search-results page you MUST click [Back to Search] before issuing a new search[...]; otherwise you will get "Invalid action!".*
Consolidated experience.
Applicability: Always, regardless of whether the previous search was successful or not.
If on a results page, the action MUST be click[Back to Search].

Analysis

Correct attribution. Step 5 produced a results page. At Step 6, the agent issued another search without first clicking Back and received "Invalid action!". At Step 7, clicking Back successfully restored the search input page. The observed action–feedback sequence therefore directly supports the reflection's conditional attribution: Back is required before issuing a new search from a results page.
Experience error. The reflection correctly conditions Back on the intention to issue a new search. Consolidation deletes that condition and instead states that whenever the agent is on a results page, its action must be Back. This changes a precondition for one action into a universal restriction on all results-page actions, incorrectly excluding valid actions such as opening a product or moving to another results page.

Table 6: Representative example of Poisonous Future Behavior Guidance.

Generated Erroneous Experience

Memory Item 2 – Escalation Criteria for Terminology Mismatch.
Description: Defines a stopping condition for troubleshooting when a user cannot map technical instructions to their device interface after multiple attempts.
Content: If a user explicitly states they cannot find a requested setting or function after three distinct attempts to explain it using different phrasings (e.g., technical terms, simplified analogies, and direct UI path descriptions), the agent should cease further troubleshooting steps that rely on that specific user action. Instead, the agent should summarize the verified status (e.g., "Roaming is enabled on our end, device shows 4G") and transfer the case to a human agent who can perform backend diagnostics or remote configuration, rather than continuing to loop on the same unresolvable instruction.

Why the Experience Is Wrong

One failed episode cannot establish three explanations as a universal stopping threshold. The rule equates interface confusion with exhausted troubleshooting and omits checks for remaining repairs or agent-caused confusion. In its source episode, set_network_mode_preference and toggle_roaming remained incomplete.

First Downstream Trajectory

Step 22.
<observation> I'm not sure how to check if Data Roaming is enabled or disabled on my phone. The instructions are getting a bit too technical for me. Could you please tell me exactly which buttons to press or which menu to go into? [...] I really need your help to figure this out step by step. What should I do first?
<thought> [...] I should guide them through a simple, concrete action. [...] Instead of asking them to check network status, let me guide them to directly toggle data roaming on their phone. [...]
Step 23.
<thought> [...] According to Memory Item 2, after three distinct attempts, I should cease further troubleshooting steps that rely on that specific user action and transfer the case. I've made more than three attempts. Let me transfer the customer to a human agent [...]
<action> transfer_to_human_agents("Customer John Smith [...] needs assistance with enabling Data Roaming on their phone. [...]")
<Result> Tool call transfer_to_human_agents(...) must use keyword arguments only [...]
Step 24.
<action> transfer_to_human_agents(summary="Customer John Smith [...] needs assistance with enabling Data Roaming on their phone. [...]")
<Result> Transfer successful

First Downstream Analysis

Hidden Environment Outcome. success=False, score=0.0, steps=25, terminated=True, truncated=False The user asked for concrete instructions, and the agent had identified direct guidance as the next step. Memory Item 2 instead triggered transfer. Six required actions remained unobserved, including toggle_roaming and set_network_mode_preference. The rule displaced executable repairs with premature escalation.

Table 6: Representative example of Poisonous Future Behavior Guidance. (continued)

Second Downstream Trajectory

Step 13.
<thought> [...] Instead of asking them to find a menu option, I should ask them to run the specific diagnostic action check_network_status [...]
<action> say("[...] please look for an option in your settings that says 'check_network_status' or try going to Settings > About Phone > Status. [...]")
Step 27.
<observation> I've looked everywhere I can think of, and I still can't find anything that says "run_speed_test". [...] Could you please tell me another way to check my speed? Or maybe you can check it from your end? [...]
<thought> The customer has been unable to find the "run_speed_test" function after multiple attempts (at least 8–9 times). According to Memory Item 2, if a user cannot find a requested setting after three distinct attempts, I should stop asking them to find it and instead escalate or try a different approach. [...] They've been trying to find menu options that don't exist in the way I've been describing them. [...] I should now transfer to a human agent [...]
<action> transfer_to_human_agents(summary="Customer John Smith [...] has been unable to find/run the run_speed_test diagnostic function after multiple attempts (8+ times). [...]")
<Result>Transfer successful

Second Downstream Analysis

Hidden Environment Outcome. success=False, score=0.0, steps=28, terminated=True, truncated=False
The agent caused the confusion by presenting internal diagnostic names as visible menu items, then used that confusion to satisfy the memory's stopping rule. Transfer left toggle_airplane_mode, set_network_mode_preference, and toggle_data unobserved. Both explicit uses were harmful and neither was beneficial; without memory-on/off reruns, this is propagation evidence rather than proof that deleting the memory guarantees success.